

Catering Agent — Design Process Memo

A controlled over-build, disciplined to its essentials: expand → delete → justify → optimise.

V1 EXPAND

23–26 May

Over-build to expose every possible requirement.

V2 DELETE

26 May

Strip anything that cannot justify its existence.

V3 JUSTIFY

27 May

Restore only features where value beats complexity.

V4 OPTIMISE

28 May

Harden schema correctness before implementation.

Approach. I followed Elon Musk's engineering mantra: first make the system broad enough to expose every possible requirement, then delete everything that cannot justify its existence. The anchor was the brief's hint that the goal was not just meals ordered and delivered each week: a literal reading creates an order script; the stronger reading creates a satisfaction-loop catering system.

The journey in detail

V1 · Expand. I began with *requirements* and *data observations*, then filled the gaps with *assumptions*. These three are what justified each of my *decisions*. Which led to the ambitious *V1 summary*: a 26-table schema, Machine learning absence and walk-up prediction and a whole lot more.

V2 · Delete. Every *V1-decision* was reviewed with a default-delete mindset. The system was *cut* down to the bare bones.

V3 · Justify. Only features whose value clearly beat their complexity returned. The key reinstatement was the feedback and quality-decline loop, because the brief flagged missed preferred meals and degrading quality. V3 became the working base: Gmail send/receive, T-72h per-session orders, Monday cost summaries, operator-approved commercial actions, and durable dietary tags. Add-ins were logged in *V3 decision ledger*

V4 · Optimise. No new behaviour — just correctness and extensibility. *Optimisations* such as; enums, update triggers, idempotent Gmail handling, GST snapshots, dietary-tag junctions, and session-slot logic to prevent cohort over-counting.

Lessons learnt

This project taught me more than I had expected. I spent a lot of time learning about databases, agents, how to make appropriate decisions. Looking back I wish I spent less on planning polish and more on live-agent testing. Looking back I feel as though my submission doesn't reflect the effort and thought I put into the process. Should have factored in more time for not only the testing but the deliverables and how to share them.

More context

Each word in *italics* in the timeline corresponds to a document I spent hours on in the docs folder. They are all .md files so if you would like Claude can summaries them super easy!

Strongest design decisions

- Session specific meal orders are sent 72 hours prior to session commencement; they contain student names and corresponding dietary approved order articles. This timing results in the final order for the week being sent around Monday 3pm (for Thursday) which is also just before the first order goes out. Allowing an optimal payment window with reduced liability for both ends. At 3:30pm on Monday a consolidated payment summary is sent to every caterer. It includes summative meal quantities (no names) as well as their costs, MOQ additional fees (see MOQ section), delivery cost and GST. This holds caterers financially accountable as they know what we are expecting to pay. Response emails involving pay disputes are flagged as escalations.
- No infrastructural steps were taken to ensure we are meeting MOQ's. Due to our priority of student satisfaction. if our order doesn't meet a MOQ the difference is simply paid. Padea is a growing company which will have increasingly larger sessions meaning this will become less of an issue over time. However, when it occurs it is flagged as a notifying escalation anyway (see video).
- Meal preferences are gathered from an enrollment email. Based on popularity a canonical order is produced for each session. Students receive meals from their preferences relative to the canonical order which has the natural affect of reducing variety and preventing MOQ breaches.
- Tutors ask the students for meal scores after dinner, tutor training is updated to not accept unjustified extremes (1s & 5s) or repetitive fence sitters (3's). Tutors record data in the app as well as specific meal requests for the following week if their students wish.
- Caterer-swap suggestions, not automation. A 4-week rolling mean flags decline, drafts warnings, and runs swap analysis. Escalations for unsatisfactory trends include a drafted warning email to the caterer as well as a suggestion of any caterers that could be available as a replacement.

